



Cairo University

Faculty of Economics and Political Science

Socio-Computing Department

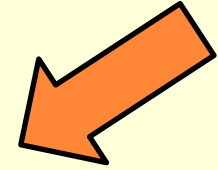
C201: Computer Applications in Social Sciences II

MODULE 1: PROGRAMMING FUNDAMENTALS

LECTURE 1_4

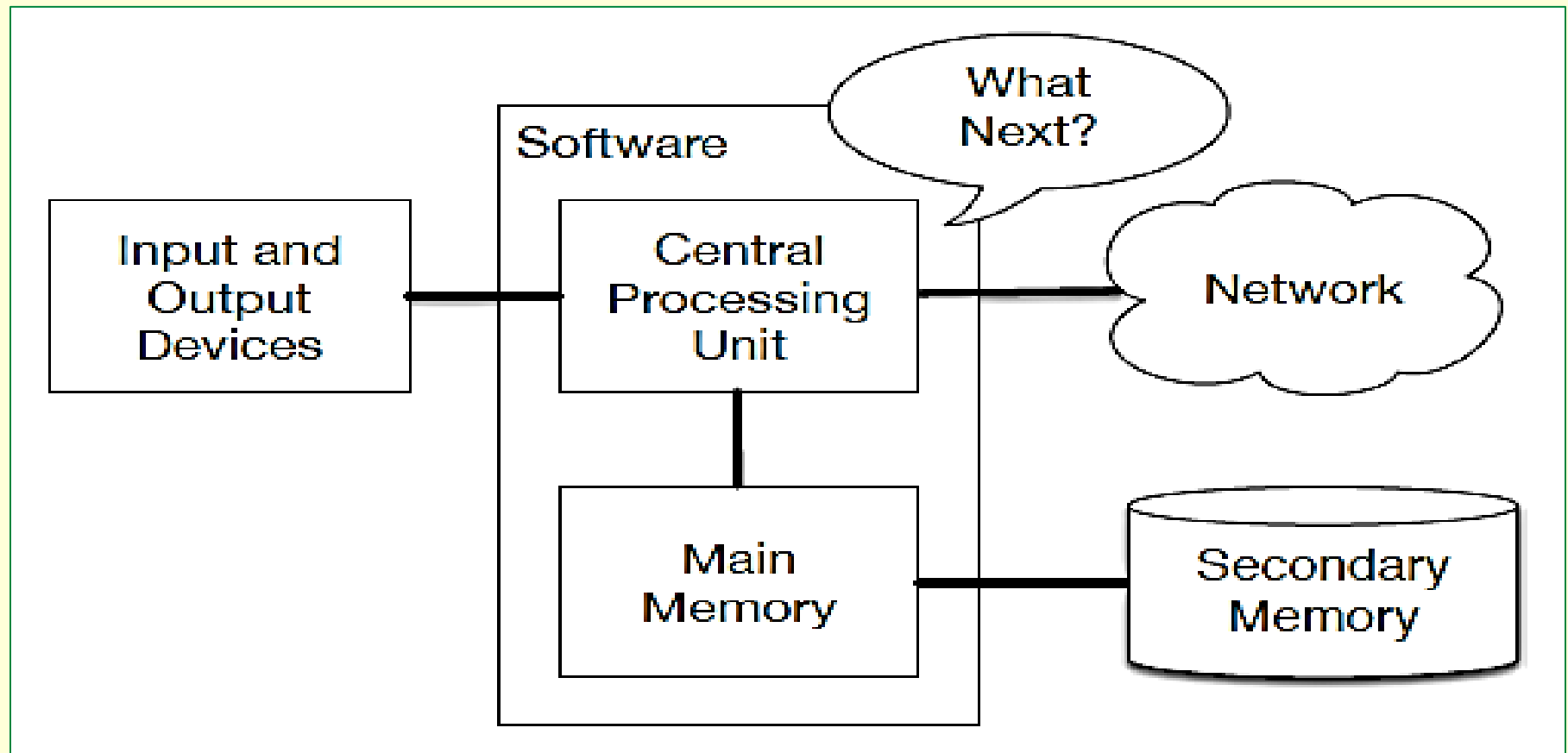
INTRODUCTION TO PROGRAMMING

OUTLINES



- ❑ **Remember:** Computer Hardware Architecture
- ❑ What is Programming? and What are the Skills the Programmer Needs to Gain
- ❑ Programming Languages
 - *What is a Programming Language?*
 - *Types of Programming Languages*
- ❑ Types of Programming Errors

Computer Hardware Architecture



Computer Hardware Architecture

Central Processing Unit



- ❑ The Central Processing Unit (or **CPU**) is the part of the computer that is built to be obsessed with “**what is next?**”
- ❑ When the clock speed (measured in GHz) of a CPU is labeled as **3.0 GHz** (Gigahertz), it means the processor completes 3 billion cycles per second (the CPU will ask “What next?” three billion times per second).
- ❑ The clock speed represents how fast the CPU processes instructions.
- ❑ A higher GHz generally means the CPU can handle tasks more quickly, but performance also depends on other factors like the
 - **The number of cores** (A multi-core CPU can handle multiple tasks simultaneously.),
 - **Architecture** (A 3.0 GHz CPU from 2024 is much more powerful than a 3.0 GHz CPU from 2010 due to improved design and efficiency.), and
 - **Cache Memory Size**. Cache memory is a small, high-speed memory located inside or **very close to the CPU**. It stores frequently accessed data and instructions to reduce the time needed to fetch them from the slower main memory (RAM).

Computer Hardware Architecture

Memory

Main Memory

- ❑ The Main Memory is used to store information that the CPU needs in a hurry.
- ❑ The main memory is nearly as fast as the CPU.
- ❑ But the information stored in the main memory **vanishes** when the computer is turned off.



Secondary Memory

- ❑ The Secondary Memory is also used to store information, but it is **much slower** than the main memory.
- ❑ The advantage of the secondary memory is that it can **store information even when there is no power** to the computer.
- ❑ Examples of secondary memory are **disk drives** or **flash memory** (typically found in USB sticks and portable music players).

Computer Hardware Architecture

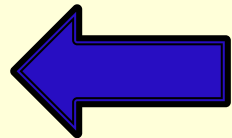
Input and Output Devices

- ❑ The Input and Output Devices are simply our screen, keyboard, mouse, microphone, speaker, touchpad, etc.
- ❑ **Networks:** These days, most computers also have a Network Connection to retrieve information over a network

They are all of the ways we interact with the computer.



Outlines

- ❑ Remember: Computer Hardware Architecture
- ❑ What is Programming? and What are the Skills
the Programmer Needs to Gain 
- ❑ Programming Languages
 - *What is a Programming Language?*
 - *Types of Programming Languages*
- ❑ Types of Programming Errors

What is Programming?

Programming means designing or creating a set of instructions to ask the computer to carry out certain jobs which normally are very much faster than human beings can do.

Two phases are required for a typical programming task

1. Problem solving phase

Define the problem and produce an ordered sequence of steps that describe solution of problem (algorithm)

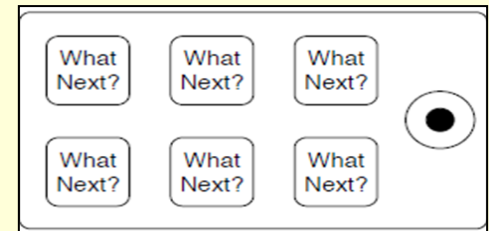
2. Implementation phase

Implement the program in some programming language

What is Programming?

- ❑ Writing programs (or programming) is a very *creative* and *rewarding* activity.
- ❑ You can write programs for many reasons, ranging from making your living to *solving a difficult data analysis problem* to having *fun* to helping someone else to *solve his own problem*.
- ❑ Computers, from laptops to cell phones, are designed to assist us by following our instructions.
- ❑ Learning to communicate in "computer language" allows us to automate routine tasks, **freeing time for more complex activities**.

For example, a computer can count words in a text and identify the most frequently used word very quickly.



What is Programming?

Computers excel at activities that people cannot, such as *processing massive volumes of data, carrying out difficult computations, automating tedious jobs, and identifying patterns in enormous datasets.*

However, humans excel at *creativity, critical thinking, and emotional intelligence*, which computers cannot replicate, making human intuition and problem-solving skills irreplaceable in many fields.

At the start of your journey, you'll be both the programmer and the user of your programs. By the end of this course, you won't be a professional programmer, but you'll have the skills to build programs that tackle simple real problems with confidence!

Programmer Skills

- ❑ A programmer directs the CPU and coordinates hardware resources—memory, storage, networks, and I/O devices—to solve problems.
- ❑ **A programmer answers the CPU’s “What next?”** question by writing instructions in advance, known as a program. Programming is the process of creating and refining these instructions.
- ❑ *First*, it is essential to understand a programming language such as Python, including its *syntax* and *structure*. This requires familiarity with its vocabulary, correct spelling of commands, and the ability to construct properly formatted statements.
- ❑ *Second*, you must effectively structure and communicate your ideas. Just as writing a story involves combining words and sentences to convey a message to the reader, programming requires skill in organizing code logically. This skill improves through practice and constructive feedback.

In programming, **our program** is the “**story**” and **the problem** you are trying to solve is the “**Story Dilemma**”.

Programmer Skills

Once you learn one programming language such as Python, you will find it much easier **to learn a second programming language** such as JavaScript or C++.

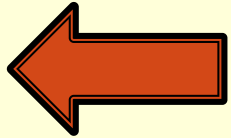
The new programming language has very *different vocabulary and grammar* but **the problem-solving skills will be the same across all programming languages**

Start reading and explaining programs,

then we write simple programs,

and then we write increasingly complex programs over time.

Outlines

- ❑ Remember: Computer Hardware Architecture
- ❑ What is Programming? and What are the Skills the Programmer Needs to Gain
- ❑ Programming Languages 
 - *What is a Programming Language?*
 - *Types of Programming Languages*
- ❑ Types of Programming Errors

WHAT IS PROGRAMMING LANGUAGE?

Programming languages are the essential tools that enable humans to **communicate** with computers. These languages serve as a bridge between human understanding and machine execution.

A programming language is a set of instructions written by a programmer to deliver instructions to the computer to perform and accomplish a task.

Like spoken languages, each language has its own

- unique *Syntax* : the rules for writing correct code (like grammar in a language).,
- *Semantics*: the meaning of the code (what it actually does).
- *Mechanisms for error handling*.

Example: two statements with different syntax and same semantic

In Python: `print("Hello, World")`

In C: `printf("Hello, World");`



Most famous programming languages

TYPES OF PROGRAMMING LANGUAGES

- Programming languages form the foundation of computer science and software development. They provide the tools and syntax required to create software on different hardware platforms.
- Over the years, programming languages have evolved to meet the increasing complexity of software systems, *the need for greater productivity*, and *the demand for higher levels of abstraction*.
- There are various types of programming languages, each designed with a specific goal in mind, which can be classified based on their level of abstraction, purpose, and interaction with the underlying hardware.
- Programming languages can be **classified** based on several key criteria, each reflecting different aspects of how they function and are used.
 - **Level of Abstraction**
 - **Execution Method**
 - **Programming Paradigm**
 - **Domain-Specific vs. General-Purpose**

TYPES OF PROGRAMMING LANGUAGES

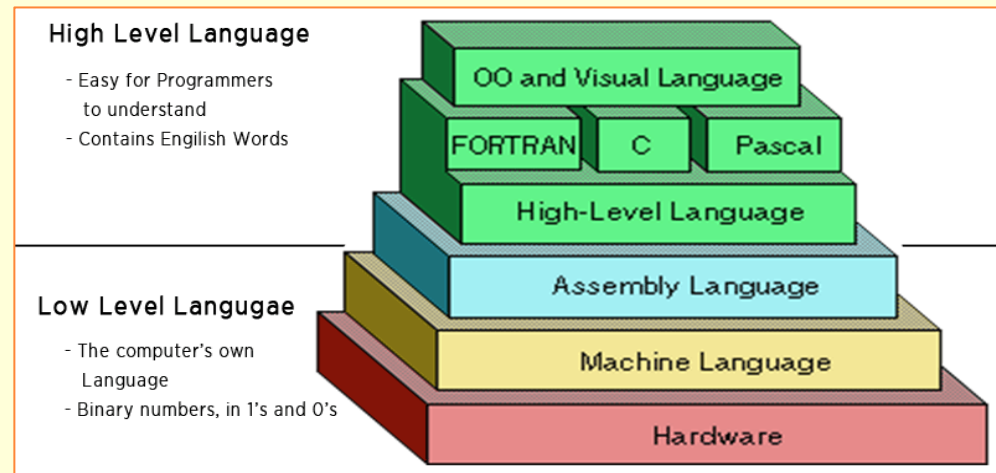
Level of Abstraction

programming languages can be categorized into low-level languages, which provide direct hardware interaction and high-level languages, which offer abstraction and user-friendly syntax.

- **Low-level languages** such as assembly and machine languages. Machine language is the lowest-level programming language, *consisting entirely of binary code (0s and 1s)* that the CPU can directly execute.

Because it is hardware-dependent and difficult for humans to read or write, programmers rarely use machine language directly. Instead, they rely on assembly language which provides a more readable representation of machine code that abstract complex machine instructions into simpler commands.

- **High-level languages** such as Python, Java, and C++ offer a higher level of abstraction, making them easier to write, read, and maintain, as they hide much of the complexity of the hardware.



TYPES OF PROGRAMMING LANGUAGES

Programming Paradigm (how the code is structured, organized, and executed)

- *Procedural Languages*: Focus on step-by-step instructions and functions (e.g., C, Pascal).
- *Object-Oriented Languages (OOP)*: Based on objects and classes for modularity and reuse (e.g., Python, Java, C++).
- *Logic-Based Languages*: Use formal logic to express computations (e.g., Prolog).

Domain-Specific vs. General-Purpose

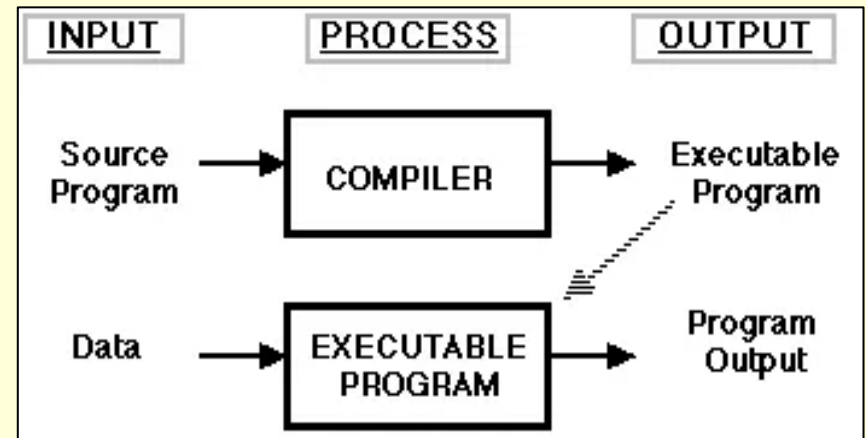
- *General-Purpose Languages*: Used for a wide variety of applications (e.g., Python, Java, C++).
- *Domain-Specific Languages*: Designed for specialized tasks, such as *SQL for databases*, *HTML for web development*, or *MATLAB for mathematical computing*.

TYPES OF PROGRAMMING LANGUAGES

Execution Method

In general, computer programs exist in High-Level Language that a human can easily understand. But computers cannot understand the same high-level language. So for computers, we have to translate them into machine language and make them readable. According to the execution method, there are three types of languages: *Compiled*, *Interpreted*, and *Hybrid*

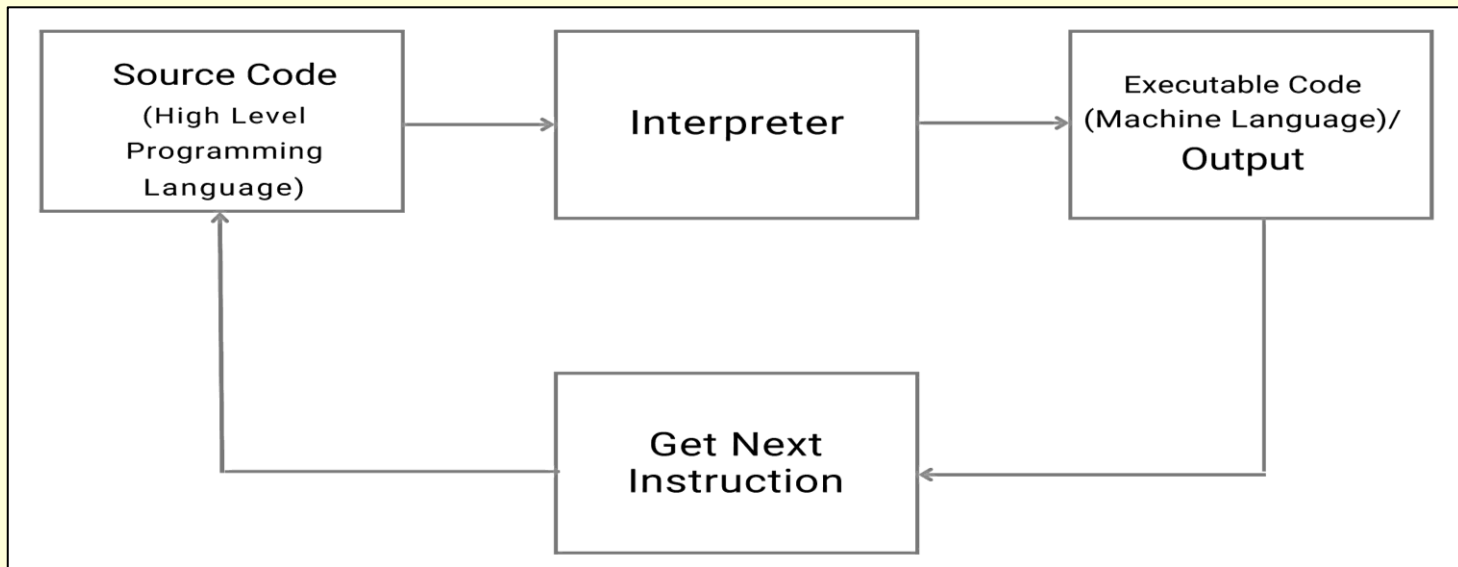
- **Compiled Languages (e.g., C, C++):** use a **compiler** translator program to convert the entire source code into machine code (i.e., 1s and 0s).
 - The compiler puts the resulting code into a file for later execution. If you have a Windows system, often these executable machine language files have the suffix “.exe” or “.dll” which stand for “executable” and “dynamic link library” respectively.
 - If you were to open an executable file in a text editor, it would look completely crazy and be unreadable.



TYPES OF PROGRAMMING LANGUAGES

Execution Method

- **Interpreted Languages (e.g., Python, JavaScript):** use an **interpreter translator program** to convert line-by-line source code into machine code at runtime, making debugging easier but often running slower



- **Hybrid Languages:** Some languages, like Java, use compilation and interpretation, compiling into an intermediate byte code that runs on a virtual machine.

DIFFERENCE BETWEEN COMPILERS AND INTERPRETERS

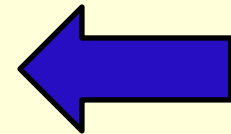
Compilers	Interpreters
They translate the entire code before execution.	They translate code line by line during execution.
Generate an executable file.	Directly executes the source code. No executable file is generated.
Typically, faster execution after compilation.	More straightforward to test and debug code interactively.
Errors are identified before execution.	Errors are identified during runtime.
Some of the compiled languages are C, C++, and C#.	Some of the interpreted languages are Python, Ruby, and PHP.

Test Your Knowledge

1. Which of the following is a key difference between a compiler and an interpreter?	A. A compiler translates code line by line, whereas an interpreter translates the entire code at once. ☒ B. A compiler translates the entire code before execution, whereas an interpreter translates code line by line C. A compiler is used only for high-level programming languages, while an interpreter is used for low-level languages. D. An interpreter requires no source code, while a compiler does.
2. What is an advantage of using an interpreter instead of a compiler?	A. Interpreted programs execute faster than compiled programs. B. Interpreters generate standalone executable files, while compilers do not. ☒ C. Interpreters provide immediate feedback, making debugging easier D. Interpreters can only be used with statically typed languages.
3. Which statement about compilers and interpreters is correct?	☒ A. An interpreted program must always be executed with the source code. B. A compiled program needs an interpreter to run. C. Compilers execute code directly without producing an intermediate file. D. Interpreters require compilation before execution.
4. Which of the following best describes how an interpreter executes a program?	A. It converts the entire source code into machine code before execution. B. It creates an independent executable file before running the program. C. It first compiles and then interprets the code simultaneously. ☒ D. It translates and executes the code line by line at runtime.
5. Which of the following is true about compiled languages?	A. They require an interpreter to run. ☒ B. They generally execute faster than interpreted languages because they are precompiled. C. They always produce human-readable intermediate code. D. They do not generate machine code before execution.

OUTLINES

- ❑ **Remember:** Computer Hardware Architecture
- ❑ What is Programming? and What are the Skills the Programmer Needs to Gain
- ❑ Programming Languages
 - *What is a Programming Language?*
 - *Types of Programming Languages*
- ❑ **Types of Programming Errors**



PROGRAM ERRORS

- We must communicate very precisely when we write Python code.
- The smallest deviation or mistake will cause Python to give up looking at your program.
- *Program errors* are issues in a program that prevent it from executing correctly.
- There are several types of program errors, each with distinct causes and impacts.
 - **Syntax Errors**
 - **Runtime Errors**
 - **Logical Errors**
 - **Semantic Errors**



TYPES OF PROGRAM ERRORS

SYNTAX ERRORS

Syntax refers to the rules and regulations for writing statements in a programming language. Each programming language has its own rules. These rules dictate how keywords, symbols, operators, and statements must be arranged to form valid instructions. *Syntax errors* occur when the code violates these rules such as missing semicolons or incorrect indentation. The compiler or interpreter detects these rules before execution.

For example,

In Python: `print("Hello, World")` (correct)

In C: the above statement is incorrect and should be written as

`printf("Hello, World");`

Common Python syntax errors include: *leaving out a keyword, putting a keyword in the wrong place, and leaving out a symbol such as a colon, comma, or brackets. Misspelling a keyword and incorrect indentation* are common syntax errors

TYPES OF PROGRAM ERRORS

SYNTAX ERRORS: PYTHON LANGUAGE EXAMPLES

Syntax Errors	Incorrect Example	Corrected Example	Explanation
Missing Colon in Control Structures	<pre>if x > 5 print("x is greater than 5")</pre>	<pre>if x > 5: print("x is greater than 5")</pre>	A colon (:) is required after 'if', 'for', 'while', etc.
Indentation Error	<pre>def greet(): print("Hello, world!")</pre>	<pre>def greet(): print("Hello, world!")</pre>	Python enforces indentation for readability.
Using Reserved Keyword as Variable	<pre>def = "Ahmed"</pre>	<pre>st_name = "Ahmed"</pre>	' def ' is a reserved keyword in Python and cannot be used as a variable name.
Incorrect String Quotation	<pre>print('Hello, world!)</pre>	<pre>print('Hello, world!') OR print("Hello, world!")</pre>	Quotes must be properly matched.
Unmatched Parentheses	<pre>print("The sum is: " + (5 + 3)</pre>	<pre>print("The sum is: " + str(5 + 3))</pre>	Parentheses must be balanced.
Missing/Extra Comma in Lists	<pre>numbers = [1, 2, 3 4, 5]</pre>	<pre>numbers = [1, 2, 3, 4, 5]</pre>	Commas must be placed correctly.
Using '=' Instead of '==' in Condition	<pre>if x = 10:</pre>	<pre>if x == 10:</pre>	' = ' is for assignment, ' == ' is for comparison.
Improper Function Definition	<pre>def say_hello(name) print("Hello, " + name)</pre>	<pre>def say_hello(name): print("Hello, " + name)</pre>	A colon (:) is required after function definition.
Unreachable Code After 'return'	<pre>def square(x): return x * x print("...")</pre>	<pre>def square(x): return x * x</pre>	Code after ' return ' won't execute.

Since you have not yet studied the rules of Python programming, these examples may seem unclear. The objective is to understand that each statement follows a specific syntax that must be adhered to. Please **revisit this slide** after a few lectures, and the concepts should become clearer.

TYPES OF PROGRAM ERRORS

RUNTIME ERRORS

If a program is syntactically correct—meaning it is free of syntax errors—the Python interpreter will execute it. However, the program may terminate unexpectedly during execution if it encounters a runtime error. A runtime error is an issue that is not detected during the parsing stage but becomes apparent only when a specific line of code is executed, such as an attempt to divide by zero.

Runtime Error	Example	Explanation
ZeroDivisionError	<code>result = 10 / 0</code>	Occurs when dividing a number by zero.
TypeError	<code>sum = "5" + 2</code>	Happens when performing an operation between incompatible types (e.g., string and integer).
ValueError	<code>num = int("abc")</code>	Raised when a function receives an argument of the correct type but an invalid value.
IndexError	<code>arr = [1, 2, 3]</code> <code>print(arr[5])</code>	Occurs when trying to access an index that does not exist in a list or tuple.
AttributeError	<code>num = 10</code> <code>num.append(5)</code>	Raised when calling an attribute or method that does not exist for an object.
FileNotFoundError	<code>file = open("missing_file.txt", "r")</code>	Occurs when attempting to open a file that does not exist.
ModuleNotFoundError	<code>import non_existent_module</code>	Happens when trying to import a module that is not installed or does not exist.

TYPES OF PROGRAM ERRORS

LOGICAL ERRORS

Logical errors occur when a program runs without crashing but produces **incorrect results** due to *imperfect code logic* (mistakes in the sequence of instructions, conditions, or loops) which can result in the program behaving unexpectedly or failing to produce the desired output. These errors are often more difficult to find compared to syntax errors as they don't crash the running of the code. Debugging logical errors requires *tracing program flow, conditions, and expected behavior*.

Logical errors	Example	Explanation
Incorrect Loop Condition	<pre>x=1 while x > 0: x = x+ 1</pre>	The condition never becomes false, causing an infinite loop.
Wrong Boolean Logic	<pre>if x > 10 and x < 5:</pre>	This condition is always false because 'x' cannot be greater than 10 and less than 5 at the same time.
Swapping Values Incorrectly	<pre>a = b b = a</pre>	Both 'a' and 'b' will have the same value, failing to swap correctly. The correct way is 'a, b = b, a'.

TYPES OF PROGRAM ERRORS

SEMANTIC ERRORS

Semantic refers to the meaning associated with the statement in a programming language. **Semantic errors** arise when the code is syntactically correct but does not do what the programmer intended. This type of error is tough to catch. For example, you can use the wrong mathematical function form. These errors also are difficult to spot as they don't crash the running of the code. Debugging semantic errors requires *checking formulas, variable meanings, and function outputs*.

Semantic Error	Example	Explanation
Using Wrong Formula	$\text{area} = \text{length} + \text{width}$ (should be $\text{length} * \text{width}$)	The formula for area is multiplication, not addition.
Wrong Order of Operations	$\text{total} = \text{tax} + \text{price} * \text{discount}$	Without parentheses, multiplication occurs first, leading to incorrect results. It should be $\text{total} = (\text{tax} + \text{price}) * \text{discount}$

Test Your Knowledge

1. Which type of error occurs when a program violates the rules of the programming language?	☒ A. Syntax Error B. Logic Error C. Runtime Error D. Semantic Error
2. What happens when a syntax error is encountered in Python?	A. The program executes but produces incorrect results B. The program crashes while running ☒ C. The program does not execute at all D. The error is ignored by the interpreter
3. Which of the following is NOT an example of a runtime error?	A. Division by zero B. Accessing an undefined variable C. Running out of memory ☒ D. A missing colon in an if statement
4. Which type of error is the hardest to detect because it does not stop program execution?	A. Syntax Error ☒ B. Logic Error C. Runtime Error D. Compilation Error
5. A semantic error means that:	☒ A. The program will not run because of incorrect syntax ☒ B. The program will run but may not produce the correct results C. The program crashes due to an illegal operation D. The interpreter automatically corrects the error

Test Your Knowledge

6. Which error type is most likely caused by using = instead of == in an if condition?	☒ A. Syntax Error B. Runtime Error C. Logic Error D. Semantic Error
7. Which of the following errors can occur even if a program is syntactically correct?	☒ A. Logic Error B. Syntax Error C. Compilation Error D. All of the above
8. What kind of error occurs when a variable is used before it is assigned a value?	A. Syntax Error ☒ B. Runtime Error C. Logic Error D. Semantic Error
9. Which statement about logic errors is correct?	A. They prevent the program from running B. They are always caused by incorrect syntax ☒ C. They can cause incorrect outputs without any error messages D. They are detected by the Python interpreter before execution
10. Which of the following is an example of a semantic error?	A. Using Print instead of print in Python B. Forgetting a closing parenthesis in an expression ☒ C. Using the wrong formula to calculate an area D. Using a variable name that starts with a number